Sorbonne Université

LU2IN013 - Projet de développement

CampusParler : Plateforme de débats

Auteurs :

Belgacem Smaali
Ali Ben Hassine
Daria Moussoki-Samba
Anaïs Chhay
Anaïs Khemiri
Siwar Daghar

Encadrants :

Nicolas Maudet
Jordan Thieyre

Résumé

CampusParler est une plateforme de débat argumenté permettant aux utilisateurs de créer et participer à des discussions structurées selon deux formats : Flash et ThinkTree. La plateforme facilite l'organisation des arguments pour et contre, le vote, et l'évaluation de la qualité argumentative des débats. Ce rapport présente le fonctionnement du logiciel, son architecture, les tests effectués ainsi qu'une analyse de son empreinte carbone.

Table des matières

Résumé **2**

1 Manuel utilisateur **5**
1.1 Introduction à CampusParler 5
1.1.1 Contexte et objectifs 5
1.1.2 Fondements théoriques 5
1.1.3 Public cible 5
1.2 Installation et démarrage 6
1.2.1 Prérequis techniques 6
1.2.2 Procédure d'installation 6
1.2.3 Démarrage du serveur 6
1.2.4 Vérification de l'installation 7
1.2.5 Résolution des problèmes courants 7
1.3 Fonctionnalités principales 7
1.3.1 Inscription et authentification 7
1.3.2 Page d'accueil 9
1.3.3 L'Agora : explorer les débats 9
1.3.4 Créer un débat 10
1.3.5 Participer à un débat 12
1.3.6 Système de vote 14

2 Architecture du projet **15**
2.1 Vue d'ensemble de l'architecture 15
2.1.1 Principes architecturaux fondamentaux 15
2.1.2 Diagramme architectural 15
2.1.3 Composants architecturaux 15
2.2 Structure du code et organisation des modules 16
2.2.1 Organisation hiérarchique des fichiers 16
2.2.2 Modèles et relations 17
2.3 Choix de developpement 17
2.3.1 Django comme framework 17
2.3.2 SQLite comme base de données 17
2.4 Algorithmes de scoring des débats 18
2.4.1 Algorithme pour les débats Flash 18
2.4.2 Algorithme pour les débats Thinktree 19
2.4.3 Comparaison et complémentarité des approches 20

3 Rapport de test **21**
3.1 Méthodologie de test . . . 21
3.2 Tests unitaires . . . 21
3.2.1 Configuration avec setUp . . . 21
3.2.2 Tests du modèle Débat . . . 21
3.2.3 Tests du modèle Argument . . . 22
3.2.4 Tests du modèle Réponse . . . 22
3.3 Tests des vues . . . 22
3.3.1 Configuration avec setUp . . . 22
3.3.2 Tests d'authentification . . . 22
3.3.3 Tests des débats Flash . . . 22
3.3.4 Tests des débats Thinktree . . . 22
3.4 Tests d'intégration . . . 23
3.4.1 Tests d'intégration de l'algorithme de score Thinktree . . . 23
3.4.2 Tests de transition d'état des débats . . . 23
3.5 Tests end-to-end (E2E) . . . 23
3.5.1 Parcours de débats Flash . . . 23
3.5.2 Parcours de débats Thinktree . . . 23
3.6 Conculsion . . . 24

4 Rapport carbone **25**
4.1 Méthodologie de mesure . . . 25
4.1.1 Choix de l'outil de mesure . . . 25
4.1.2 Protocole expérimental . . . 25
4.1.3 Aspects mesurés . . . 25
4.2 Résultats des mesures par fonctionnalité . . . 26
4.2.1 Tableau récapitulatif des émissions . . . 26
4.2.2 Visualisation graphique . . . 26
4.2.3 Interprétation . . . 26
4.3 Analyse comparative des fonctionnalités . . . 27
4.3.1 Comparaison par catégorie de fonctionnalités . . . 27
4.3.2 Extrapolation des resultats . . . 27

Chapitre 1

Manuel utilisateur

1.1 Introduction à CampusParler

1.1.1 Contexte et objectifs

CampusParler est une plateforme de débat argumenté développée dans le cadre du cours LU2IN013 - Projet de développement à Sorbonne Université.

Face à la polarisation croissante des espaces de débats/discussion en ligne et à la difficulté d'y mener des échanges structurés, CampusParler propose une approche alternative axée sur la qualité argumentative plutôt que sur la viralité des contenus. La plateforme vise à encourager des débats constructifs où la force des arguments prévaut sur leur nombre.

1.1.2 Fondements théoriques

CampusParler s'appuie sur deux modèles de structuration des débats :

— **Le modèle Flash** : Une structure binaire classique (arguments pour/contre) avec un système de scoring global calculant la force relative des deux positions.
— **Le modèle ThinkTree** : Une structure arborescente inspirée de la plateforme Kialo, où chaque argument peut être soutenu ou contesté individuellement, créant ainsi un arbre argumentatif complexe.

Ces deux approches font appel à différents types d'argumentation et d'algorithmes de scoring qui seront détaillés plus tard.

1.1.3 Public cible

La plateforme s'adresse principalement :

— Aux étudiants de Sorbonne Université. Il est possible d'y soumettre des projets locaux et proposer différentes idées concernant le campus. Ou bien de débattre de sujets plus larges ou philosophiques dans le contexte de la vie étudiante à l'échelle du pays voire du monde.

1.2 Installation et démarrage

1.2.1 Prérequis techniques

L'installation de CampusParler nécessite les éléments suivants :
— Python 3.8 ou supérieur
— Pip (gestionnaire de paquets Python)
— Git
— SQLite (inclus avec Django)
— Navigateur web

1.2.2 Procédure d'installation

Récupération du code source

```
git clone <URL_du_dépôt>
```

Création d'un environnement virtuel

```
python -m venv venv
source venv/bin/activate  % Sur Linux/Mac
venv\Scripts\activate     % Sur Windows
```

Installation des dépendances

```
pip install -r requirements.txt
```

Initialisation de la base de données

```
cd campusparler
python manage.py migrate
```

Création d'un utilisateur administrateur (facultatif)

```
python manage.py createsuperuser
```

1.2.3 Démarrage du serveur

En mode développement

```
python manage.py runserver
```

Le serveur sera accessible à l'adresse : http://127.0.0.1:8000/

En mode test (avec configuration isolée pour les rapports carbone par exemple)

Utilisez un fichier de configuration spécifique :

```
python manage.py runserver --settings=settings_test
```

En mode "normal"

```
python manage.py runserver
```

1.2.4 Vérification de l'installation

Accédez à `http://127.0.0.1:8000/` dans votre navigateur. La page d'accueil de CampusParler devrait s'afficher.

1.2.5 Résolution des problèmes courants

Problème	**Solution**
Erreur `ModuleNotFoundError`	Vérifiez que l'environnement virtuel est bien activé
Erreur de migration	Supprimez le fichier `db.sqlite3` et relancez `python manage.py migrate`
Problème d'accès à la base de données	Vérifiez les permissions du dossier contenant `db.sqlite3`

1.3 Fonctionnalités principales

La navigation sur CampusParler est simple et intuitive. Le menu principal se trouve en haut de chaque page et vous permet d'accéder aux différentes sections du site.

1.3.1 Inscription et authentification

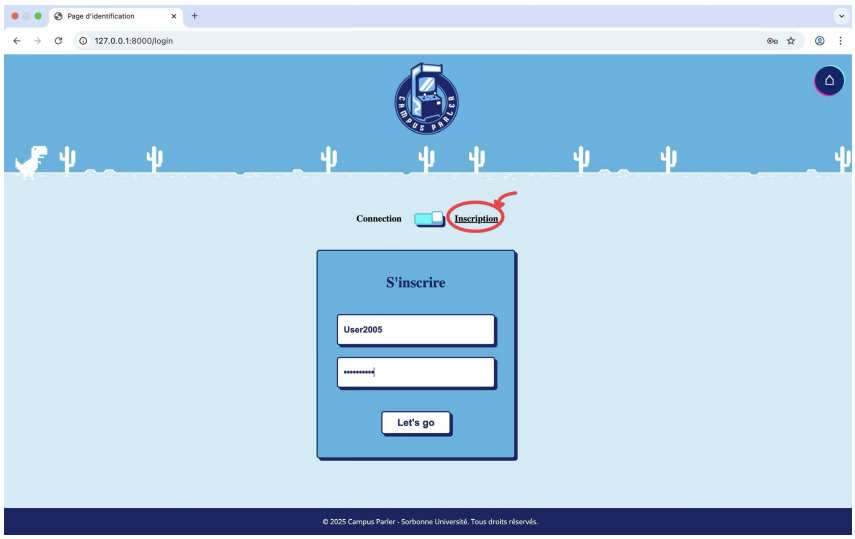
CampusParler propose un système d'authentification complet permettant :

— La création de nouveaux comptes avec nom d'utilisateur et mot de passe
— La connexion à un compte existant
— La déconnexion

Le système utilise l'authentification native de Django.

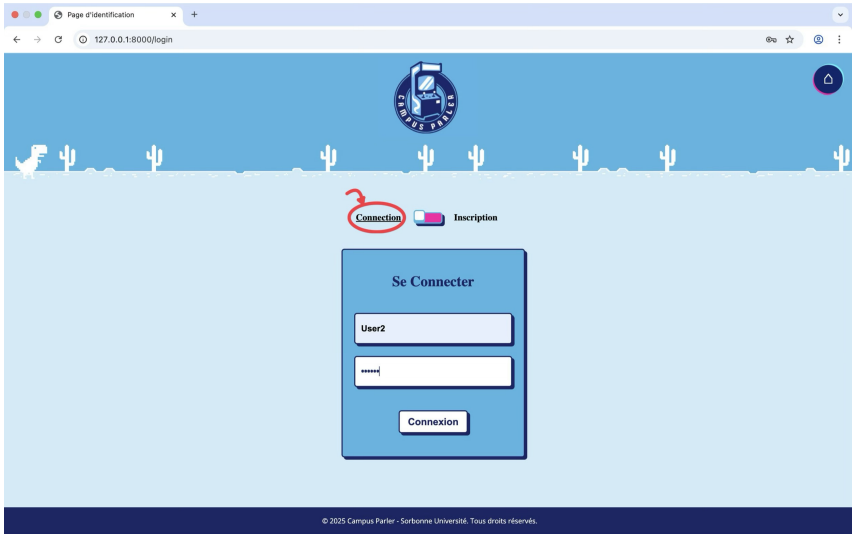
Pour s'inscrire :

— Cliquez sur le bouton **"S'identifier"** en haut à droite de la page d'accueil
— Sélectionnez **"Créer un compte"**
— Remplissez les informations requises
— Validez votre inscription



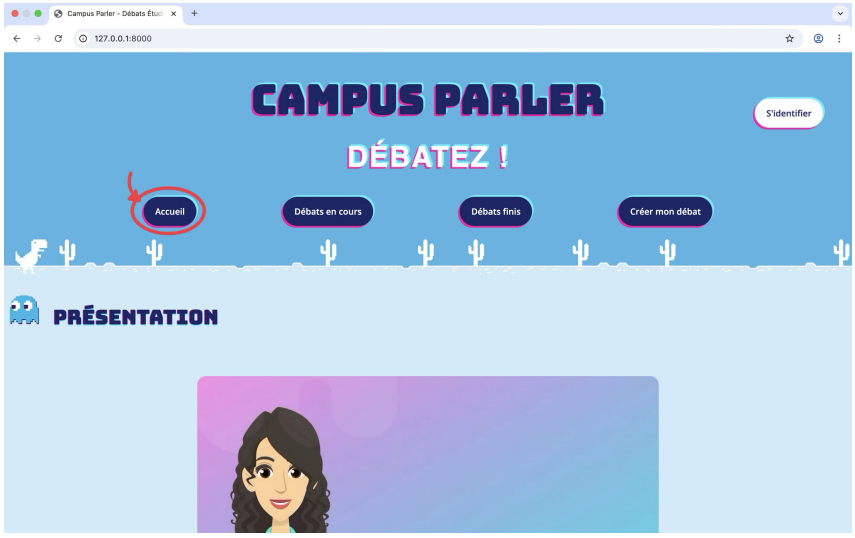
Pour se connecter :

— Cliquez sur le bouton **"S'identifier"** en haut à droite de la page d'accueil
— Entrez votre identifiant et votre mot de passe
— Validez votre connexion



1.3.2 Page d'accueil

La page d'accueil vous présente la plateforme CampusParler et son objectif principal à travers une vidéo. Elle se présente ainsi :

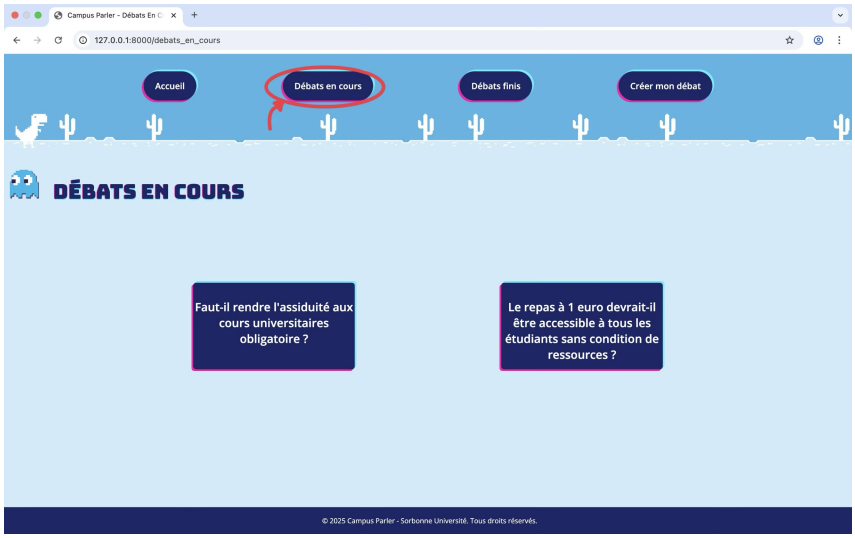


1.3.3 L'Agora : explorer les débats

La plateforme distingue automatiquement les débats en cours des débats terminés :
— Un débat est considéré "en cours" pendant 5 jours après sa création.
— Après cette période, il passe automatiquement en statut "terminé".

Débats en cours

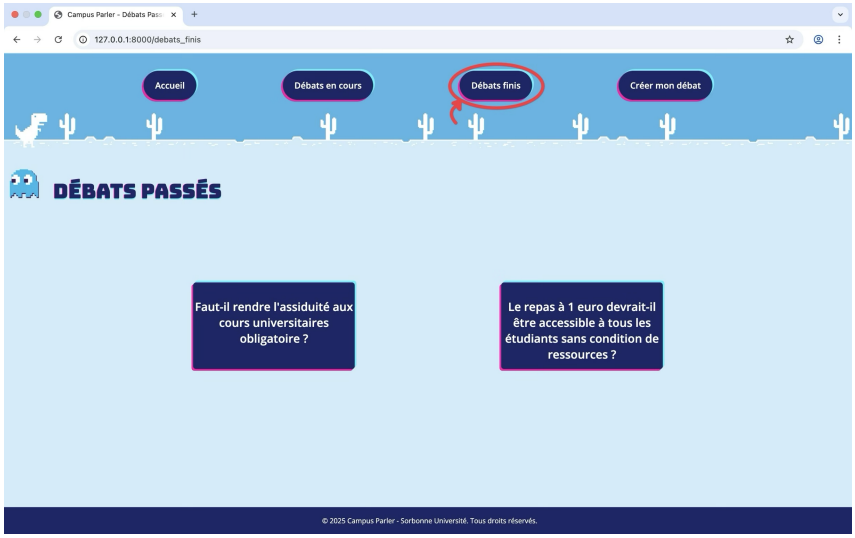
Cette page affiche tous les débats actuellement actifs auxquels vous pouvez participer.



Chaque débat est représenté par une vignette indiquant son titre. Pour participer à un débat, il vous suffit de cliquer sur la vignette correspondante.

Débats finis

Les débats terminés restent accessibles en lecture seule, permettant de consulter :
— Le score final du débat
— L'ensemble des arguments et contre-arguments



1.3.4 Créer un débat

Pour lancer un nouveau débat, suivez ces étapes :
— Cliquez sur le bouton **"Créer mon débat"** dans le menu principal
— Choisissez le type de débat que vous souhaitez créer :



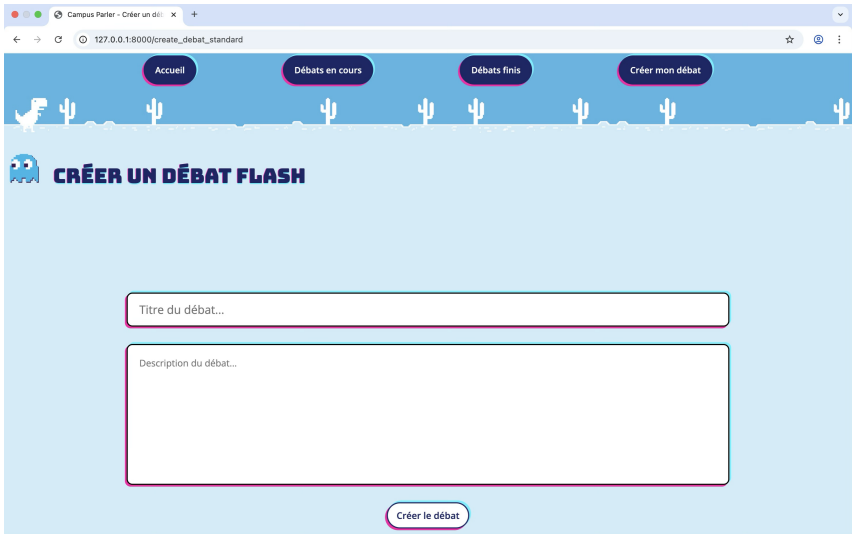
CampusParler propose deux formats de débats différents :

Débat Flash

Format simplifié avec des arguments pour et contre. Les réponses aux arguments ne sont pas reliées entre elles, permettant une discussion concise.

Pour créer un débat Flash :

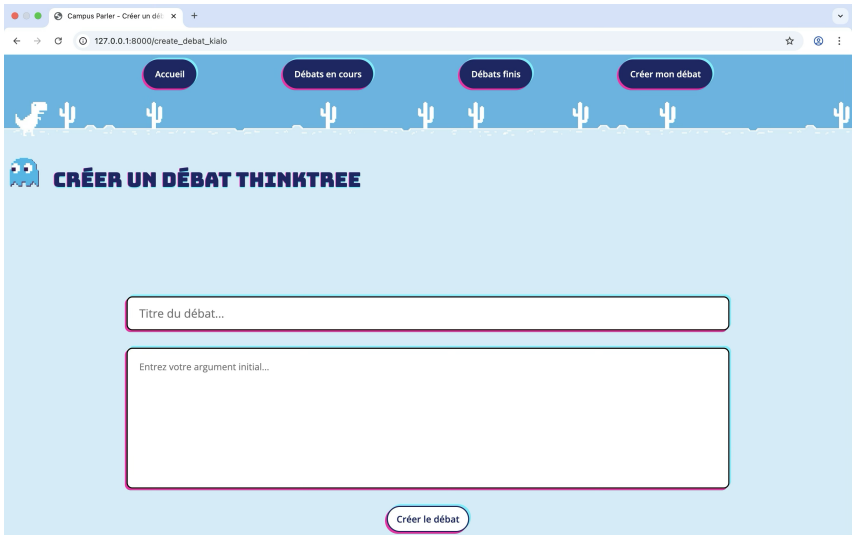
— Sélectionnez **"Débat Flash"**
— Remplissez les champs requis :
  — **Titre du débat** : donnez un titre clair et accrocheur à votre sujet de débat
  — **Description du débat** : présentez brièvement le contexte et les enjeux de votre sujet
— Cliquez sur **"Créer le débat"**



Débat ThinkTree

Format hiérarchique avec des arguments imbriqués. La structure en arbre permet une discussion plus approfondie et nuancée.

Pour créer un débat ThinkTree, suivez les mêmes étapes que pour un débat Flash, mais en sélectionnant **"Débat ThinkTree"** comme format.



1.3.5 Participer à un débat

Une fois dans un débat, vous pouvez consulter les arguments existants et contribuer à la discussion.

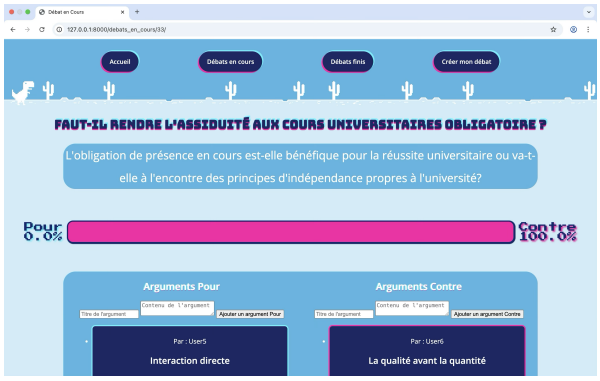
Débat Flash

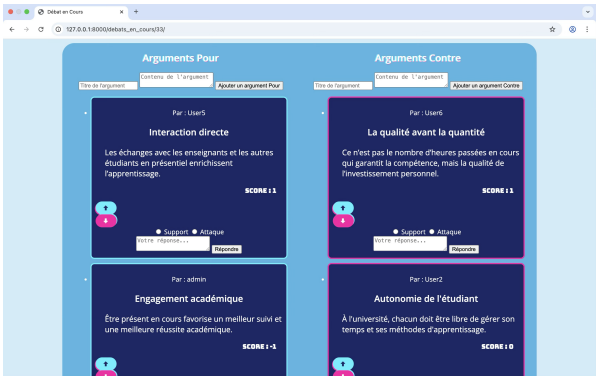
Les débats Flash organisent les arguments en deux catégories distinctes :
— Arguments "Pour" (position favorable à la thèse du débat)
— Arguments "Contre" (position défavorable à la thèse du débat)

Chaque argument comprend :
— Un titre concis
— Un contenu développé
— Une position (pour/contre)
— Un score calculé à partir des votes

Les arguments peuvent recevoir des réponses directes qui :
— Sont associées à un argument spécifique
— Peuvent soutenir ou contester l'argument parent
— Sont prises en compte dans le calcul du score global

Dans un débat Flash, la page se présente ainsi :





Pour participer :

— Consultez les arguments existants
— Utilisez les boutons ↑ et ↓ pour voter sur les arguments
— Ajoutez votre propre argument en remplissant le formulaire correspondant

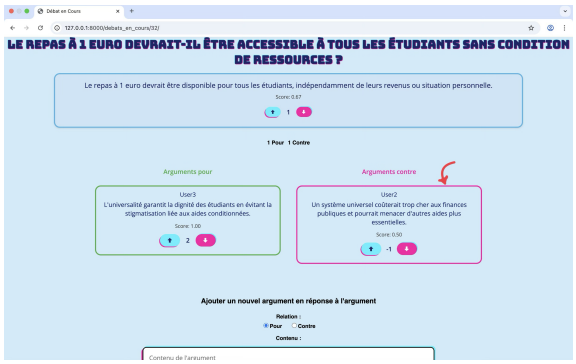
Débat ThinkTree

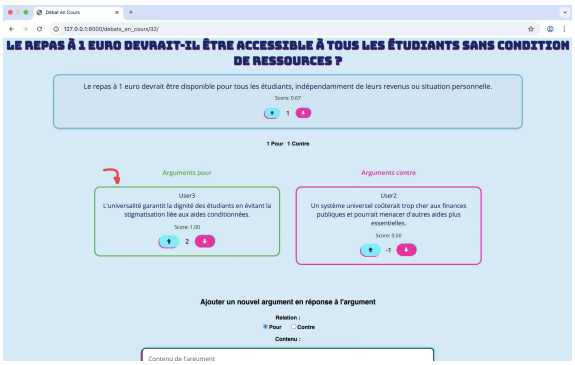
Contrairement au modèle Flash :

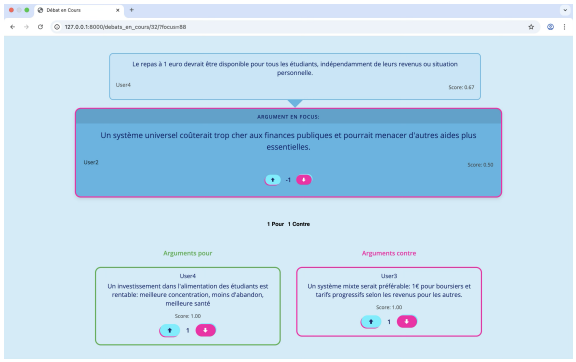
— Chaque argument est directement lié à un argument parent
— Les arguments peuvent être "Pour" (relation=1) ou "Contre" (relation=-1)

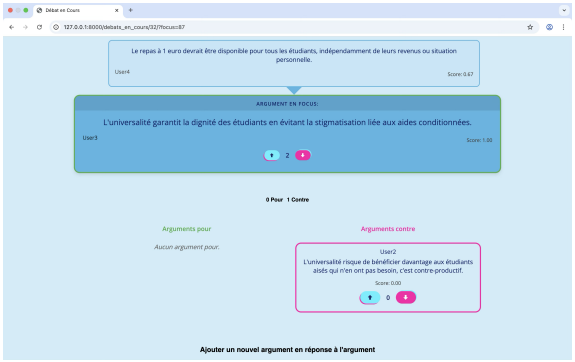
L'interface permet de :

— Visualiser l'argument central (focus) et ses arguments directs
— Naviguer dans l'arborescence en cliquant sur un argument
— Suivre le chemin depuis la racine jusqu'à l'argument en focus









Pour participer :
— Parcourez l'arborescence des arguments
— Votez sur les arguments existants si souhaitez
— Répondez à un argument spécifique en cliquant sur **"Répondre"**
— Ajoutez un nouvel argument en précisant sa relation au parent

1.3.6 Système de vote

Interaction des utilisateurs

Les utilisateurs peuvent interagir avec les arguments via des votes positifs et négatifs.

Chaque utilisateur peut voter une seule fois par argument. Les votes sont pris en compte en temps réel dans le calcul des scores (cf ....). Le système empêche le double vote sur un même élément.

Visualisation des scores

Les scores sont présentés de manière intuitive :
— Pour les débats Flashs : pourcentage global pour/contre
— Pour les débats Thinktree : score individuel de chaque argument dont l'argument initial du débat.

Chapitre 2

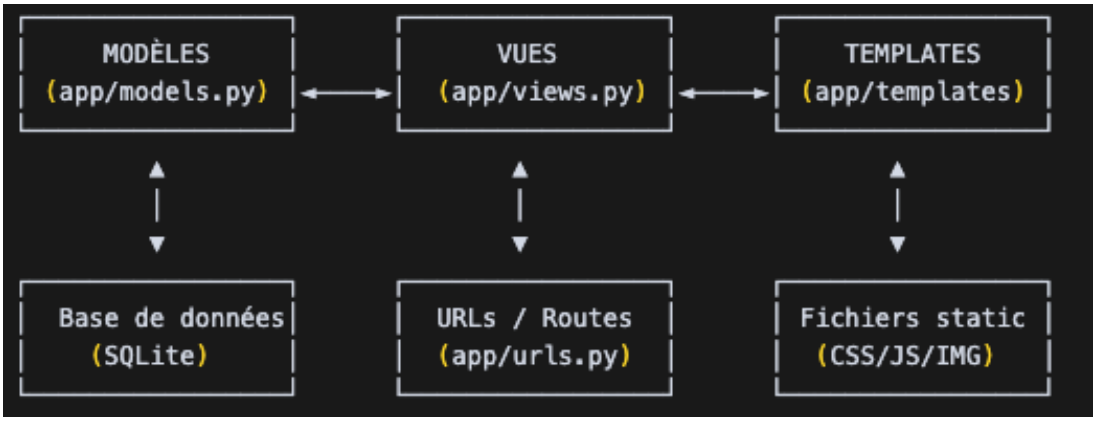
Architecture du projet

2.1 Vue d'ensemble de l'architecture

2.1.1 Principes architecturaux fondamentaux

CampusParler repose sur une architecture MVT (Modèle-Vue-Template), propre au framework Django.

2.1.2 Diagramme architectural



2.1.3 Composants architecturaux

1. **Modèles**
   — Définissent la structure des données
   — Implémentent les relations entre les entités
   — Assurent la validation et l'intégrité des données

2. **Vues**
   — Traitent les requêtes HTTP entrantes
   — Transmettent les données aux templates
   — Implémentent les algorithmes de scoring

3. **Templates**
   — Définissent la structure HTML des pages
   — Intègrent le moteur de template Django pour l'affichage dynamique

4. **URLs**
   — Associent les adresses URL aux vues correspondantes
   — Définissent les paramètres capturés dans les URLs
   — Structurent la hiérarchie des pages

2.2 Structure du code et organisation des modules

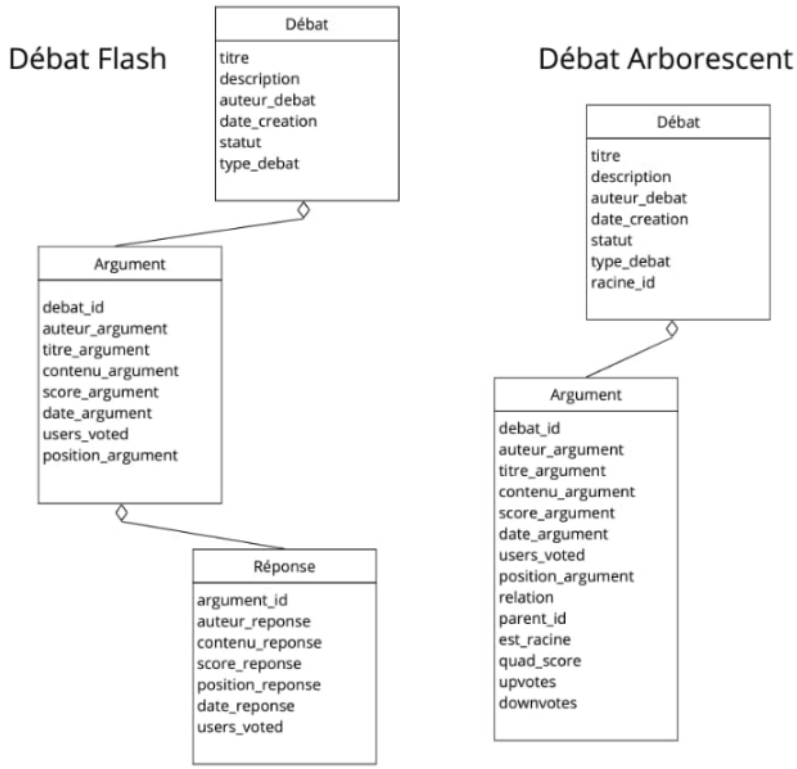
2.2.1 Organisation hiérarchique des fichiers

La structure du projet respecte les conventions Django tout en intégrant des éléments spécifiques à notre application :



2.2.2 Modèles et relations

Le diagramme entité-relation ci-dessous représente les principales classes et leurs relations :



2.3 Choix de developpement

2.3.1 Django comme framework

Notre projet s'appuie sur Django pour plusieurs raisons pédagogiques et pratiques :

- **Facilité d'apprentissage** : Tous les contributeurs sont à l'aise et on deja utilisé python donc ca nous paraissait évident de choisir le framework Django
- **Documentation riche** : Plusieurs tutoriels disponible sur le site officiel de Django permettant une prise en main rapide du framework
- **Admin** : Interface d'administration disponible à l'adresse : `http://127.0.0.1:8000/admin`

2.3.2 SQLite comme base de données

Le projet utilise SQLite pour différentes rasion :

- **Simplicité** : Aucune installation ou configuration fastidieuse nécessaire car deja implementé de base dans Django
- **Portabilité** : Facilite le partage du projet entre étudiants car c'est base de données très légère.

2.4 Algorithmes de scoring des débats

La conception de CampusParler s'articule autour de deux modèles distincts de débats, chacun doté de son propre système d'évaluation. Cette dualité constitue un choix de développement délibéré, motivé par notre volonté d'explorer différentes approches.

2.4.1 Algorithme pour les débats Flash

Les débats Flash dans CampusParler organisent les arguments en deux camps opposés (pour/contre). L'algorithme de scoring doit déterminer lequel des deux camps présente l'argumentation la plus convaincante, en tenant compte non seulement des arguments principaux mais aussi des réponses qu'ils reçoivent.

Approche simplifiée et intuitive

Contrairement aux débats Thinktree qui utilisent une structure arborescente complexe, les débats Flash adoptent une approche plus directe et accessible. Nous avons privilégié un algorithme de scoring intuitif.

Formulation mathématique

Soit :
— $A_p = \{a_1, a_2, \ldots, a_n\}$ l'ensemble des arguments pour
— $A_c = \{a_1, a_2, \ldots, a_m\}$ l'ensemble des arguments contre
— $s(a)$ le score initial d'un argument (provenant des votes)
— $R(a) = \{r_1, r_2, \ldots, r_k\}$ l'ensemble des réponses à l'argument a
— $s(r)$ le score d'une réponse r (provenant des votes)
— $p(r) \in \{vrai, faux\}$ la position d'une réponse (vrai=camp pour et faux=contre)

Pour chaque argument $a \in A_p$ du camp "Pour", nous calculons son score effectif $s_e(a)$ en tenant compte de ses réponses :

$$s_e(a) = s(a) + \sum_{r \in R(a)} \begin{cases} s(r) & \text{si } p(r) = vrai \\ -s(r) & \text{si } p(r) = faux \end{cases} \tag{2.1}$$

De même, pour chaque argument $a \in A_c$ du camp "Contre", le score effectif est :

$$s_e(a) = s(a) + \sum_{r \in R(a)} \begin{cases} s(r) & \text{si } p(r) = faux \\ -s(r) & \text{si } p(r) = vrai \end{cases} \tag{2.2}$$

Nous calculons ensuite le score total de chaque camp en additionnant les scores effectifs de leurs arguments respectifs, avec une garantie de non-négativité :

$$S_p = \sum_{a \in A_p} s_e(a)) \tag{2.3}$$

$$S_c = \sum_{a \in A_c} s_e(a)) \tag{2.4}$$

Le score final du débat est alors calculé comme la proportion du score du camp "Pour" par rapport au score total des deux camps :

$$S_{dbat} = \begin{cases} 0.5 & \text{si } S_p + S_c = 0 \\ \frac{S_p}{S_p + S_c} & \text{sinon} \end{cases} \tag{2.5}$$

Cette formule donne un score entre 0 et 1, où 0.5 représente un équilibre parfait, les valeurs supérieures à 0.5 indiquent un avantage pour le camp "Pour", et les valeurs inférieures à 0.5 un avantage pour le camp "Contre".

2.4.2 Algorithme pour les débats Thinktree

Les débats Thinktree organisent les arguments en structure arborescente où chaque argument peut être soutenu ou contesté par d'autres arguments.

Principes de la sémantique Quad

La sémantique Quad repose sur quatre principes fondamentaux :

1. Chaque argument possède un score intrinsèque basé sur les votes directs
2. La force de support d'un argument augmente avec la qualité de ses arguments de soutien
3. La force d'attaque d'un argument diminue en fonction de la qualité de ses arguments d'attaque
4. Le score final combine les forces d'attaque et de support de manière équilibrée

Formulation mathématique

Soit :
— α un argument dans l'arbre
— $S^+(\alpha) = \{s_1, s_2, \ldots, s_n\}$ l'ensemble des arguments qui soutiennent α
— $S^-(\alpha) = \{a_1, a_2, \ldots, a_m\}$ l'ensemble des arguments qui attaquent α
— $up(\alpha)$ le nombre de votes positifs pour α
— $down(\alpha)$ le nombre de votes négatifs pour α

Le score initial de l'argument, basé uniquement sur ses votes directs, est calculé par :

$$s^I(\alpha) = \begin{cases} 0.5 & \text{si } up(\alpha) + down(\alpha) = 0 \\ \frac{up(\alpha)}{up(\alpha) + down(\alpha)} & \text{sinon} \end{cases} \tag{2.6}$$

Nous définissons ensuite deux fonctions fondamentales :

1. Force d'attaque - La mesure dans laquelle les attaquants réduisent la force de l'argument :

$$f^-(\alpha) = s^I(\alpha) \cdot \prod_{a \in S^-(\alpha)} (1 - s^f(a)) \tag{2.7}$$

où $s^f(a)$ est le score final de l'argument attaquant a.

2. Force de support - La mesure dans laquelle les supporteurs renforcent l'argument :

$$f^+(\alpha) = 1 - (1 - s^I(\alpha)) \cdot \prod_{s \in S^+(\alpha)} (1 - s^f(s)) \tag{2.8}$$

où $s^f(s)$ est le score final de l'argument supporteur s.

Le score final $s^f(\alpha)$ d'un argument est calculé en combinant ces forces selon les cas suivants :

$$s^f(\alpha) = \begin{cases} s^I(\alpha) & \text{si } S^+(\alpha) = \emptyset \text{ et } S^-(\alpha) = \emptyset \\ f^-(\alpha) & \text{si } S^+(\alpha) = \emptyset \text{ et } S^-(\alpha) \neq \emptyset \\ f^+(\alpha) & \text{si } S^+(\alpha) \neq \emptyset \text{ et } S^-(\alpha) = \emptyset \\ \frac{f^+(\alpha)+f^-(\alpha)}{2} & \text{si } S^+(\alpha) \neq \emptyset \text{ et } S^-(\alpha) \neq \emptyset \end{cases} \tag{2.9}$$

Traitement global au niveau du débat

Pour calculer l'ensemble des scores d'un débat, nous procédons de manière systématique :

1. Récupération de tous les arguments associés au débat
2. Calcul récursif du score de chaque argument avec partage du cache
3. Mise à jour des scores dans la base de données

2.4.3 Comparaison et complémentarité des approches

La coexistence de ces deux algorithmes et type de débat au sein de l'application permet d'observer des différences intéressantes :

— Le scoring des débats Flash favorise une vision globale
— Le scoring des débats Thinktree permet une évaluation plus nuancée
— Les débats Flash sont plus accessibles et directs
— Les débats Thinktree permettent une exploration plus approfondie des "nuances" argumentatives

Cette dualité offre aux utilisateurs la flexibilité de choisir le format le plus adapté à leur sujet de débat.

Chapitre 3

Rapport de test

3.1 Méthodologie de test

Pour tester Campus Parler, on a adopté une approche structurée basée sur les outils de Django. Les tests sont organisés en trois catégories principales :

— **Tests unitaires** : Validation des modèles et fonctions individuelles
— **Tests d'intégration** : Vérification des interactions entre composants
— **Tests end-to-end (E2E)** : Simulation de parcours utilisateur complets

On a utilisé le framework de test intégré à Django (`django.test`) qui facilite la création d'environnements de test isolés avec une base de données temporaire.

Pour lancer les tests, on utilise la commande standard :

```
python3 manage.py test app.tests
```

3.2 Tests unitaires

3.2.1 Configuration avec setUp

Chaque classe de test utilise une méthode `setUp` qui prépare l'environnement de test. Par exemple, dans la classe `DebatTest`, la méthode `setUp` crée un utilisateur de test qui sera utilisé dans tous les tests de la classe. De même, dans `ArgumentTest`, la méthode `setUp` crée non seulement un utilisateur mais aussi un débat de test, ce qui permet de tester les arguments.

3.2.2 Tests du modèle Débat

Les tests du modèle Débat vérifient :
— `test_debat_flash` : Vérifie la création et les propriétés des débats Flash
— `test_debat_thinktree` : Test la création des débats Thinktree et leur relation avec un argument racine

Ces tests s'assurent que les attributs comme `titre`, `description`, `auteur_debat` et `statut` sont correctement initialisés.

3.2.3 Tests du modèle Argument

Pour le modèle Argument, on vérifie :
— `test_argument_standard` : Test la création et propriétés d'un argument classique
— `test_argument_thinktree` : Test le fonctionnement des arguments dans une structure arborescente
— `test_calcul_score` : On test si le calcul du score basé sur les votes est correct

3.2.4 Tests du modèle Réponse

Les tests pour le modèle Réponse incluent :
— `test_proprietes_reponse` : Vérification des propriétés d'une réponse à un argument
— `test_gestion_votes_reponse` : Test de la gestion des votes sur les réponses

3.3 Tests des vues

3.3.1 Configuration avec setUp

Les méthodes `setUp` des tests de vue sont plus complexes. Par exemple, dans `DebatFlashViewsTest`, on :
— Crée un client HTTP pour simuler les requêtes
— Crée et connecte un utilisateur
— Génère les URLs utiles pour les tests
— Crée un débat de test avec ses caractéristiques

Cette préparation permet ensuite d'exécuter des tests ciblés dans un contexte similaire à notre projet.

3.3.2 Tests d'authentification

Dans la classe `AuthViewsTest`, on vérifie :
— `test_inscription_utilisateur` : Test le processus d'inscription d'un nouvel utilisateur
— `test_connexion_utilisateur` : Test la connexion et la redirection vers la page d'accueil

3.3.3 Tests des débats Flash

La classe `DebatFlashViewsTest` contient :
— `test_creation_debat_flash` : Test la création d'un nouveau débat Flash
— `test_affichage_details_debat_flash` : Vérification de l'affichage des détails
— `test_ajout_argument_debat_flash` : Test l'ajout d'un argument à un débat existant
— `test_vote_sur_argument` : Test le fonctionnement du système de vote

3.3.4 Tests des débats Thinktree

Pour les débats Thinktree (`DebatThinktreeViewsTest`), on teste :

— `test_creation_debat_thinktree` : On test la création simultanée d'un débat et de son argument racine
— `test_affichage_details_debat_thinktree` : On test l'affichage de la structure arborescente
— `test_ajout_argument_thinktree` : Test l'ajout d'un argument enfant à la racine

3.4 Tests d'intégration

3.4.1 Tests d'intégration de l'algorithme de score Thinktree

La classe `KialoScoreIntegrationTest` vérifie le fonctionnement de l'algorithme de calcul de score pour les débats Thinktree :
— `test_mise_a_jour_scores` : Vérifie que la mise à jour des scores fonctionne pour tous les arguments de l'arborescence
— `test_impact_nouvel_argument` : Test que l'ajout d'un nouvel argument influence correctement le score de son parent

3.4.2 Tests de transition d'état des débats

La classe `DebatStatusTransitionTest` vérifie les transitions d'état des débats :
— `test_transition_automatique_termine` : Test que les débats sont automatiquement marqués comme terminés après 5 jours
— `test_affichage_debat_termine` : Vérifie qu'un débat terminé apparaît bien dans la liste des débats finis et non dans celle des débats en cours

3.5 Tests end-to-end (E2E)

Les tests E2E simulent des parcours utilisateur complets, de bout en bout, pour s'assurer que l'application fonctionne correctement dans son ensemble.

3.5.1 Parcours de débats Flash

La classe `DebatFlashE2ETest` :
— `test_parcours_complet_debat_flash` : Test un parcous utlisateur complet avec un debat Flash

3.5.2 Parcours de débats Thinktree

La classe `DebatThinktreeE2ETest` vérifie :
— `test_parcours_complet_debat_thinktree` : Test un parcous utlisateur complet avec un debat Thinktree

3.6 Conculsion

L'ensemble des tests couvre les fonctionnalités essentielles de Campus Parler à tous les niveaux.

Au cours de nos tests, on a identifié certaines limites qui nécessitent une attention particulière :

- **Nombre d'arguments dans un débat** : Les tests ont montré que la performance reste correcte jusqu'à environ 100 arguments par débat. Au-delà, le chargement de la page et le calcul des scores peuvent devenir plus lents.
- **Algorithme de calcul des scores** : Pour les débats Thinktree complexes (>50 arguments), le temps de calcul pour la mise à jour de tous les scores peut dépasser 2 seconde.

Ces limites identifiées en soit ne constituent pas des problèmes pour l'usage prévu de l'application.

Chapitre 4

Rapport carbone

4.1 Méthodologie de mesure

4.1.1 Choix de l'outil de mesure

Dans le cadre de notre porjet, nous avons decider de choisir le module CodeCarbon au lieu de pyJoules pour mesurer l'empreinte carbonne pour plusieurs raisons :

— **Compatibilité multiplateforme** : Contrairement à pyJoules qui présente des limitations sur macOS, CodeCarbon fonctionne de manière fiable sur tous les systèmes d'exploitation, ce qui était essentiel pour notre équipe.
— **Conversion directe en CO_2** : CodeCarbon convertit automatiquement les mesures énergétiques en équivalent CO_2, intégrant des facteurs régionaux d'intensité carbone du réseau électrique

4.1.2 Protocole expérimental

Notre protocole de mesure a été conçu pour refléter l'utilisation réelle de l'application tout en isolant chaque fonctionnalité :

1. **Configuration de test** : Serveur Django lancé avec les paramètres de test (–settings=config.settings_test) pour ne pas encombrer la base de données principale.
2. **Isolation des fonctionnalités** : Chaque fonctionnalité a été mesurée séparément grâce à la fonction `mesurer_fonction` qui encapsule le tracker d'émissions de CodeCarbon
3. **Itérations multiples** : Pour chaque fonctionnalité, nous avons effectué 10 itérations afin d'obtenir des résultats statistiquement significatifs et de limiter l'impact des variations ponctuelles

4.1.3 Aspects mesurés

Notre analyse a couvert l'ensemble des fonctionnalités principales de CampusParler :

— **Navigation et accès** : Chargement des différentes pages de l'application
— **Authentification** : Processus d'inscription, connexion et déconnexion
— **Débats Flash** : Création, consultation et interaction

— **Débats Thinktree** : Création, consultation et interaction avec la structure arborescente

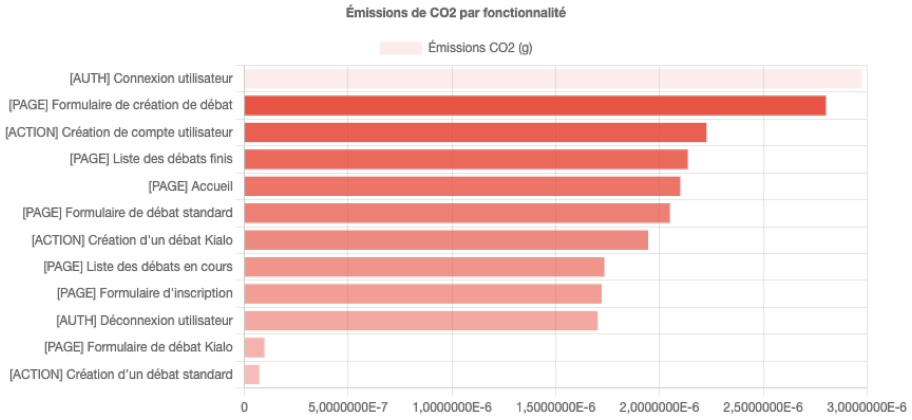
4.2 Résultats des mesures par fonctionnalité

4.2.1 Tableau récapitulatif des émissions

Détails par fonctionnalité

Fonctionnalité	URL	Émissions CO2 (g)	Itérations	% du total
[AUTH] Connexion utilisateur	/login	0.00000297	10	13.79%
[PAGE] Formulaire de création de débat	/create_debat	0.00000280	10	12.99%
[ACTION] Création de compte utilisateur	/register	0.00000223	10	10.33%
[PAGE] Liste des débats finis	/debats_finis	0.00000214	10	9.91%
[PAGE] Accueil	/	0.00000210	10	9.74%
[PAGE] Formulaire de débat standard	/create_debat_standard	0.00000205	10	9.51%
[ACTION] Création d'un débat Kialo	/save_debat_kialo	0.00000195	10	9.02%
[PAGE] Liste des débats en cours	/debats_en_cours	0.00000173	10	8.05%
[PAGE] Formulaire d'inscription	/register	0.00000172	10	7.99%
[AUTH] Déconnexion utilisateur	/logout	0.00000170	10	7.89%
[PAGE] Formulaire de débat Kialo	/create_debat_kialo	0.00000010	10	0.45%
[ACTION] Création d'un débat standard	/save_debat_standard	0.00000007	10	0.33%

4.2.2 Visualisation graphique



4.2.3 Interprétation

Les résultats de nos mesures révèlent des valeurs d'émission très faibles, de l'ordre de 10^{-6} grammes de CO_2 par requête. Ces valeurs extrêmement basses s'expliquent par plusieurs facteurs :

— **Environnement de test local** : L'application s'exécutait sur une machine locale sans la complexité d'une infrastructure complexe
— **Utilisateur unique** : Nos tests simulaient un seul utilisateur, alors qu'une application avec une infrastructure à grande échelle servirait simultanément de nombreux utilisateurs
— **Absence de réseau étendu** : Les communications réseau étaient limitées à la boucle locale (localhost), éliminant les émissions liées au transit des données sur Internet
— **Durée limitée** : Nos mesures concernaient des opérations ponctuelles plutôt que l'utilisation continue sur de longues périodes

4.3 Analyse comparative des fonctionnalités

4.3.1 Comparaison par catégorie de fonctionnalités

L'analyse comparative de l'empreinte carbone des différentes fonctionnalités révèle plusieurs tendances significatives :

— **Opérations d'authentification** : Les fonctions d'authentification, notamment la connexion utilisateur, présentent une empreinte relativement élevée par rapport aux autres fonctionnalités. Cela s'explique par les opérations cryptographiques nécessaires à la vérification des mots de passe et à la gestion des sessions.

— **Création versus consultation** : Les opérations de création (débats, arguments) sont généralement plus coûteuses en carbone que les opérations de simple consultation, ce qui est cohérent avec le nombre supérieur d'écritures en base de données qu'elles impliquent.

— **Débats Thinktree versus Flash** : La création et la manipulation des débats Thinktree présentent une empreinte légèrement supérieure à celle des débats Flash, reflétant la complexité accrue de la structure arborescente et des calculs de score récursifs.

— **Chargement des pages** : Les pages contenant des éléments interactifs complexes (comme la visualisation des débats Thinktree) sont plus gourmandes en ressources que les pages statiques.

4.3.2 Extrapolation des resultats

Bien que nos mesures aient été réalisées dans un environnement local, nous pouvons extrapoler ces résultats pour estimer l'empreinte carbone dans un environnment réel :

— En considérant une université de taille moyenne (20 000 étudiants), avec 5% d'utilisateurs actifs quotidiens effectuant en moyenne 10 opérations par jour, l'empreinte annuelle resterait de l'ordre de quelques kilogrammes de CO_2.

— L'infrastructure d'hébergement (serveurs, réseau, refroidissement) représenterait probablement une part bien plus importante de l'empreinte totale que l'exécution du code lui-même.